

Introduction

Stone Game V shows up on interview lists for a specific reason: it's one of those problems where the intuitive first move is provably wrong, and figuring out *why* teaches you more about dynamic programming than a dozen easier problems combined. Interviewers like it because it tests whether you can resist a plausible-but-wrong greedy strategy and instead recognize the recursive structure hiding underneath: every split creates two smaller versions of the exact same problem.

If you've been solving DP problems by pattern-matching templates without really internalizing why the state is defined the way it is, this problem will expose that gap fast — and fix it. By the end, you'll understand not just how to solve Stone Game V, but how to recognize the broader family of "split into two pieces, solve each piece, combine" problems that this pattern covers.

Problem Overview

You're given a row of stones, each with a positive value. In one move, you pick a point in the row and split it into two non-empty, contiguous groups — left and right. You compare the sum of each group. Whichever group has the **smaller** sum gets added to your score, and that smaller group is then discarded entirely. The larger group survives and becomes the new row for the next move. If both groups have equal sums, you choose which one to keep, and the other is scored and discarded.

You keep doing this — split, score the smaller side, discard it, keep going on the survivor — until only one stone is left. The question: what's the maximum total score you can end up with, over all possible sequences of splits?

The size of the row shrinks by roughly half (at best) every round, but the *order* in which you carve it up matters enormously, which is exactly what makes this a DP problem rather than a simple simulation.

Example

Take the row `[6, 2, 3, 4, 5, 5]`. Played optimally, the maximum achievable score is **18**.

To see why order matters, zoom into just the first three stones: `[6, 2, 3]`.

- **Even split** (after the 6-2 boundary isn't possible with 3 stones evenly, so split after index 1): piles are `[6, 2]` = 8 and `[3]` = 3. The smaller pile (3) scores, and since it's a single stone, nothing more can happen. Total: **3**.

- **Lopsided split** (right after the first stone): piles are $[6] = 6$ and $[2, 3] = 5$. The smaller pile (5) scores, but it's not a single stone — it can be split again into 2 and 3, scoring another 2. Total: 7.

Same three stones, but the "unbalanced" cut beats the "balanced" one by 4 points. That's the trap this problem is built around.

Intuition

The instinct most people bring to this problem is: keep the two sides as close in size as possible, so you never lose a huge chunk of value in one move. It feels safe. The example above shows it isn't even locally optimal, let alone globally.

Here's the reframe that unlocks the problem: every split just carves the *current segment* into two smaller segments, and each of those segments is a smaller instance of the exact same question — "what's the best score obtainable from this segment alone?" That's the signal to define the problem in terms of a **segment**, not the whole array.

So define $dp[i][j]$ = the maximum score obtainable starting from the segment of stones from index i to j , played to completion. If you know the best answer for every smaller segment, you can build up the answer for the full array by trying every possible split point and taking the best one.

One subtlety trips people up here: it's tempting to fold the *score accumulated so far* into the cache key — something like $dp[i][j][score]$. Don't. The score you've already banked from earlier splits doesn't change which future split is optimal inside this segment — it's just added on top afterward. Keying only on (i, j) is what keeps the state space at $O(n^2)$ instead of exploding.

Brute Force Solution

Idea: Recursively try every split point for every segment, without any caching. For a segment $[i, j]$, try each split point k , compute the left and right sums, recurse into whichever side is smaller (adding its score plus itself), and take the max over all k .

Algorithm: 1. Base case: a segment of one stone scores 0. 2. For each split point k between i and j , compute $leftSum$ and $rightSum$. 3. If $leftSum < rightSum$, score is $leftSum + solve(i, k)$. 4. If $rightSum < leftSum$, score is $rightSum + solve(k+1, j)$. 5. If equal, score is $leftSum + \max(solve(i, k), solve(k+1, j))$. 6. Take the max over all k .

Advantages: Simple to reason about, directly mirrors the problem statement.

Disadvantages: Without memoization, the same segments get recomputed exponentially many times — this blows up well past what's feasible for n up to 500.

Complexity: Time is exponential (roughly $O(2^n)$ in the worst case), space is $O(n)$ for the recursion stack.

Optimal Solution

The optimal solution is the brute force idea, but with two upgrades: memoize on (i, j) so every segment is solved exactly once, and precompute **prefix sums** so any range sum is an $O(1)$ lookup instead of an $O(n)$ re-add.

Step 1 — Prefix sums. Build an array `prefix` where `prefix[i+1]` holds the sum of all stones from index `0` to `i`. Then the sum of the range `[i, j]` is simply `prefix[j+1] - prefix[i]`.

Step 2 — DP table. Loop over every segment length, then every starting index, then every split point inside that segment:

Segment Split point Left sum Right sum Smaller side scores

<code>[1, 4, 2]</code> after idx 0	1	6	1 (left)
<code>[1, 4, 2]</code> after idx 1	5	2	2 (right)

For `[1, 4, 2]`: splitting after index 0 gives left=1, right=6 → left is smaller, score 1. Splitting after index 1 gives left=5, right=2 → right is smaller, score 2. The best of these is 2, so `dp` for this segment is 2.

Step 3 — Combine. For each split, the score is the smaller side's sum plus whatever `dp` already computed for that smaller side (since it can keep being split further). Take the max over every split point in the segment.

Step 4 — Tie handling. If `leftSum == rightSum`, you get to choose which side survives — so take the side whose *own* `dp` value is larger, since that's the one worth keeping alive for future splits. Miss this and you silently undercount the answer.

The final answer is `dp[0][n-1]` — the best score obtainable from the entire array.

Python Code

```
from functools import lru_cache
from itertools import accumulate
```

```

def stone_game_v(stone_values: list[int]) -> int:
    n = len(stone_values)
    prefix = [0] + list(accumulate(stone_values))

    def range_sum(i: int, j: int) -> int:
        # sum of stone_values[i..j], inclusive
        return prefix[j + 1] - prefix[i]

    @lru_cache(maxsize=None)
    def dp(i: int, j: int) -> int:
        if i == j:
            return 0

        best = 0
        for k in range(i, j):
            left_sum = range_sum(i, k)
            right_sum = range_sum(k + 1, j)

            if left_sum < right_sum:
                best = max(best, left_sum + dp(i, k))
            elif right_sum < left_sum:
                best = max(best, right_sum + dp(k + 1, j))
            else:
                # tie: keep whichever side's own dp value is bigger
                best = max(best, left_sum + max(dp(i, k), dp(k + 1, j)))

        return best

    return dp(0, n - 1)

```

Code Walkthrough

- `prefix = [0] + list(accumulate(stone_values))` : builds the running-total array so `prefix[x]` is the sum of the first `x` stones.
- `range_sum(i, j)` : turns any range sum into a single subtraction using the prefix array.
- `dp(i, j)` : the memoized recursive function representing the best score for segment `[i, j]`. `lru_cache` handles the memoization on `(i, j)` automatically — exactly the state we decided *not* to pollute with the running score.
- The base case `i == j` is a single stone: it can't be split further, so it contributes 0 additional score.
- The loop over `k` tries every valid split point inside the segment.
- The three branches handle left-smaller, right-smaller, and tied cases respectively, matching the rule described above.
- `dp(0, n - 1)` at the end kicks off the recursion for the full array.

Dry Run

Row: `[6, 2, 3, 4, 5, 5]`.

Focus on the segment `[6, 2, 3]` first (indices 0–2). Best split is after index 0: left=6, right=5 (sum of 2+3). Right is smaller, scoring 5, and it can be split again into 2 and 3 for another 2 points. `dp(0, 2) = 7`.

Now the full array. Split after index 2: left sum = 6+2+3 = 11, right sum = 4+5+5 = 14. Left is smaller, so this split scores `11 + dp(0, 2) = 11 + 7 = 18`. That matches the target answer for this row — and checking other split points confirms none beats 18.

Complexity Analysis

- **Time:** $O(n^2)$ segments, each scanning $O(n)$ split points $\rightarrow O(n^3)$ overall. With n up to 500, that's roughly 125 million operations — well inside typical time limits, since each operation is a cheap arithmetic comparison.
- **Space:** $O(n^2)$ for the memoization table plus $O(n)$ for the prefix array $\rightarrow O(n^2)$.
- **Why it's correct:** every segment's answer only depends on smaller segments' answers, computed once and reused, so the recursion terminates and never redoes work. The prefix sums don't change the algorithm's correctness — they just remove redundant summation.

Alternative Solutions

A bottom-up (iterative) tabulation version replaces the recursive `lru_cache` calls with explicit nested loops over segment length, start index, and split point. It has identical $O(n^3)$ time and $O(n^2)$ space, and avoids Python's recursion overhead, which can matter for n close to 500. The recursive version above is easier to read and reason about; the iterative version is marginally faster in practice. There's no known solution better than $O(n^3)$ with a simple DP formulation — some competitive programmers use monotonic-stack tricks to shave this to $O(n^2 \log n)$ or $O(n^2)$, but that's well beyond what's expected in an interview setting.

Edge Cases

- **Single stone (`n == 1`):** no splits possible, answer is 0.
- **Two stones:** exactly one split, the smaller value is the answer (or either, if equal, since both act identically going forward).

- **All equal values:** every split is a tie, so the tie-break rule (keep the side with the larger `dp`) gets exercised constantly — a good stress test for that branch.
- **Strictly increasing or decreasing values:** the "always split off the single smallest element" strategy tends to look attractive here, but the DP will find better splits that keep bigger useful segments alive.
- **Maximum size input ($n = 500$) with large stone values:** verify the prefix sums don't overflow assumptions in other languages — not an issue in Python, but worth noting if porting the solution.

Common Mistakes

1. **Folding the running score into the memo key** (`dp[i][j][score]`), which explodes the state space and is unnecessary since the banked score doesn't affect future optimal splits.
2. **Getting the tie-break rule backwards** — discarding the side with the larger `dp` value instead of keeping it, which silently undercounts the answer.
3. **Re-summing ranges with a loop instead of prefix sums**, which turns an $O(n^3)$ solution into an $O(n^4)$ one and can time out on the largest inputs.
4. **Off-by-one errors in prefix sum indexing**, especially forgetting the leading `0` in the prefix array or mismatching inclusive/exclusive range boundaries.
5. **Assuming the evenly-split strategy is always optimal**, which fails on cases as small as three stones, as shown in the walkthrough example.

Interview Questions

- Why is `dp[i][j]` needed instead of just `dp[i]`?
- What's the total number of (i, j) states, and how does that combine with the split-point loop to produce the overall time complexity?
- How would you convert the memoized recursive solution into an iterative bottom-up one?
- Why doesn't the running score need to be part of the memoization key?
- Can you improve the time complexity beyond $O(n^3)$, and what would that require?

Similar Problems

- **Burst Balloons (LeetCode 312):** another interval DP problem where you pick an order of operations over a range and combine sub-results — same `dp[i][j]` shape.
- **Minimum Cost to Merge Stones (LeetCode 1000):** also splits/merges a row of stones with cost accumulation, directly related in spirit.

- **Predict the Winner (LeetCode 486):** a simpler two-player interval DP that builds the same intuition for `dp[i][j]` before tackling Stone Game V.
- **Matrix Chain Multiplication (classic DP):** the canonical "try every split point in a range" problem that Stone Game V's structure closely mirrors.

Key Takeaways

Stone Game V is fundamentally about recognizing that a problem which repeatedly splits input into two pieces reduces to solving every possible sub-segment once and combining results — the textbook interval DP pattern. The greedy "split evenly" instinct fails because it optimizes for safety in a single move rather than total accumulated score across all future moves. Prefix sums remove the redundant arithmetic, and getting the tie-break rule right is what separates a working solution from one that silently returns a slightly wrong answer.

Watch the Video

For the full walkthrough with the DP table filled in by hand, the failed greedy example traced live, and the brute-force-versus-optimal timing comparison, watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series covering LeetCode's daily challenge with a focus on building real intuition — not just memorizing a template — behind each solution. Every episode works through the wrong-but-tempting approach first, shows exactly where it breaks, and then rebuilds the correct solution from first principles.

Call To Action

If this walkthrough helped the DP click, give the video a like on YouTube and subscribe so you don't miss the next daily problem. For deeper write-ups like this one delivered straight to your inbox, subscribe to the Substack. Drop a comment with the problem you want tackled next, and share this with anyone prepping for coding interviews.

Quick Review

Stone Game V: what pattern/tell signals interval DP here?

Choosing a split point over a range where the result depends on subrange scores → think interval/range DP, $dp[i][j]$ over a segment.

Time complexity of the $O(i,j)$ DP for Stone Game V?

$O(n^3)$ naive (n^2 states $\times$ $O(n)$ split), reducible to $O(n^2)$ with prefix-sum + monotonic tricks.

Space complexity of the $dp[i][j]$ table here?

$O(n^2)$ for the score table, plus $O(n)$ prefix sums for range-sum queries.

Trickiest bug when filling $dp[i][j]$ by hand?

Forgetting you take the MAX over all split points k , and on a tie you may choose either half — missing this undercounts the best score.

Why does 'split down the middle' fail as a greedy instinct?

You always lose the smaller-or-equal half, so an uneven split can preserve more value for future rounds than an even one — must try every k .

How does $dp[i][j]$ differ from a whole-row running total?

$dp[i][j]$ answers 'best score achievable from just this segment,' recursing into sub-segments, not accumulating one global sum left-to-right.